

Week Six

Sagemaker, Techniques, and System Prompts

Prompt Engineering

Mick McQuaid

mcq@utexas.edu

University of Texas at Austin

25 Feb 2025

Week Six

Agenda

- Presentations: Kylie, Aditya
- News
- Review whatialreadyknow (Ishwari)
- eC review
- m1 review
- Haystack (again)
- Techniques
- System prompts
- Work time

Presentations

News

The Batch

⟨ pause to look at this week's edition ⟩

WhatIAlreadyKnow (Ishwari)

eC review

- What did you learn from this exercise?
- One thing Ishwari and I noticed was limited documentation of prompt tuning, so did you learn a lot about prompt tuning?
- No parameter tuning that I know of
- Almost everyone used the same 20 tweets, probably due to uniform Python scripts with 42 as the random seed
- Everyone reported accuracy results but not everyone reported latency and cost, which is a problem
- Only a couple of people reported a score composed of a weighted linear combination of accuracy, latency, and cost
- Most people just eyeballed the results and said that the most accurate model and prompt were the best, but some acknowledged that the cost and latency were important

- Where cost and latency gave conflicting results, some people reported that one should take precedence, while others said that it depends on the user priority
- The best accuracy reported by anyone was 95 percent, but it was not well-documented—the best well-documented accuracy was 85 percent
- The lowest winner on accuracy was 60 percent
- Many people did not include a table of accuracy, latency, and cost, which was disappointing
- One person included prompt type (CoT, “regular”) in their table! Great!
- Some people used VADER as a benchmark, generally finding it worse than the best prompt / model combinations; one person used a different benchmark from formulabot, which scored 95 percent, several used text2data, one used TextBlob

- There was a lot of variety in the detail level of results
- Some very thoughtful conclusions, e.g., about the effect of CoT on latency and accuracy
- I really appreciated the documentation of the selection of two best prompts, especially one that used the blockquote feature of Quarto
- A wide variety of models were used, including gpt-4o-mini, gemini-2.0-flash-001, llama-2-70-b-chat-hf, llama-3.3-70b-versatile, o3-mini-high, mistral-large, gemini-pro, claude 2.1, gpt 3.5, Claude 3.5 Sonnet:beta, claude-3.5-haiku, Cohere v2, DeepSeek v2, Sonar Pro, Llama 8192, Grok 3 DeepSearch
- Many people included screenshots of tables from Agenta instead of creating tables on their own, which was disappointing
- A key learning should have been how to compare models and prompts, some mentioned that in the conclusion

- One unexpected learning was a bit of data wrangling to ensure things like column name matching
- Some people reported discovering how LLMs interpret questions differently
- There is a difference between using curly braces and not using them in the labels of code chunks, for example `{python}` should cause code to be run, but not including the curly braces should cause code to be displayed but not run
- One person got a confidence score with the benchmark tool, which highlights a limitation of the prompting techniques used by nearly everyone—a confidence score would have been possible, although it would have required more effort and skill (one person did it)
- One person selected zero-shot and few-shot prompts to compare, which was a good idea

- No one used the feature of Quarto that allows you to save results from code chunks to display in markdown text (some people noticed that they got different results in different runs)
- One person got better results from one model with less context rather than more
- Some students investigated the individual tweets and the “ground truth” associated with them, finding it sometimes misleading
- One interesting table was the one that showed the 20 tweets alongside the sentiments reported as ground truth and three models, allowing a very detailed comparison
- One person tried to use VADER from NLTK without success
- Good insights about the difficulty of neutral tweets compared to extremes of positive and negative (model is looking for cues and reads the absence of cues as neutral), as well as insights the problem of URLs in tweets

- It's valuable to learn more about Python, Markdown, and AI in general, as one person noted
- People spent a wide variety of time on this assignment!
- Some people experienced inexplicable errors in Agenta and soldiered on, I appreciate that!
- One person got uppercase responses and didn't remedy that, while others then asked explicitly for lowercase responses
- One person used gpt-o3-mini to generate prompts!
- One person looked at MMLU and HellaSwag metrics to choose models!
- The best report, in my view, was the one with the most extensive and illuminating conclusion, which I have put on Canvas (several others were close, though)
- I really appreciate efforts to make the report more readable! This can be facilitated by studying Quarto a bit

- “even a minor clarification in instructions can lead to better consistency and accuracy ... highlights the need for structured prompt experimentation”
- One person wrote a lengthy report in the style of an academic paper, perhaps assisted by genAI

More on Techniques

Last time, we continued with more prompting techniques. Now we'll move on to decomposition techniques and others. Again, we're referring to Schulhoff et al. ([2024](#)).

Decomposition

- *explicitly decomposing the problem into subproblems*
- Least-to-Most Prompting
- Plan-and-Solve Prompting
- Tree-of-Thought Prompting
- Recursion-of-Thought Prompting
- Program-of-Thoughts
- Faithful Chain-of-Thought
- Skeleton-of-Thought
- Metacognitive Prompting

Ensembling

- *using multiple prompts to solve the same problem, then aggregating the results, for example, by majority vote*
- Demonstration Ensembling
- Mixture of Reasoning Experts
- Max Mutual Information Method
- Self-Consistency
- Universal Self-Consistency
- Meta-Reasoning over Multiple CoTs
- DiVeRSe
- Consistency-based Self-Adaptive Prompting

More Ensembling

- Universal Self-Adaptive Prompting
- Prompt Paraphrasing

Self-Criticism

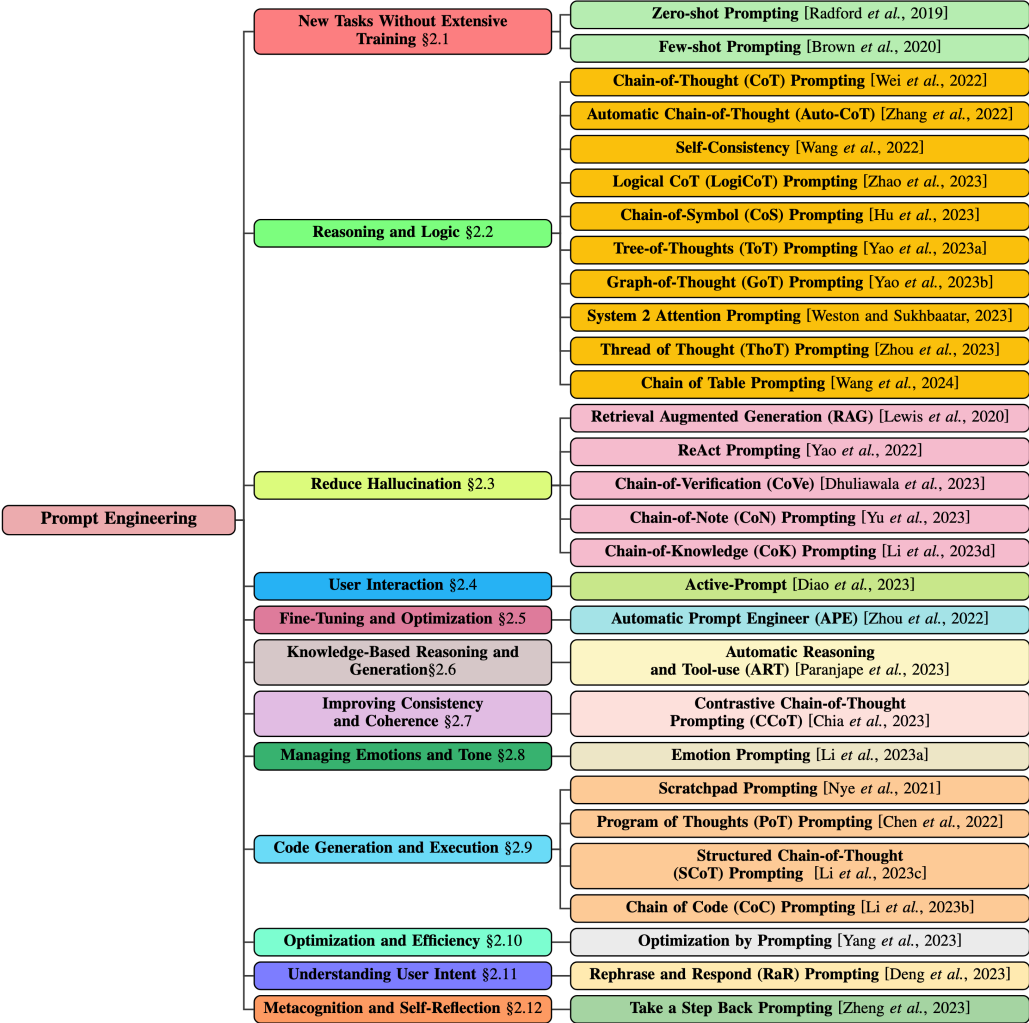
- *prompting the model to critique its own output*
- Self-Calibration
- Self-Refine
- Reversing Chain-of-Thought
- Self-Verification
- Chain-of-Verification
- Cumulative Reasoning

Prompt Engineering Techniques and Goals

Prompt Engineering Techniques

- Meta Prompting
- AutoPrompt
- Automatic Prompt Engineer
- Gradientfree Instructional Prompt Search
- Prompt Optimization with Textual Gradients
- RLPrompt
- Dialogue-comprised Policy-gradient-based Discreet Prompt Optimization

Tree of Prompt Engineering Techniques from Sahoo et al. (2024)



Goals of Prompt Engineering, Sahoo et al. (2024), 1 of 2

- New tasks without extensive training
- Reasoning and logic
- Reduce hallucination
- User interaction
- Fine-tuning and optimization
- Knowledge-based reasoning and generation
- Improving consistency and coherence

Goals of Prompt Engineering, Sahoo et al. (2024), 2 of 2

- Managing emotions and tone
- Code generation and execution
- Optimization and Efficiency
- Understanding User Intent
- Metacognition and Self-Reflection

Automatic Prompt Optimization, Wan et al. (2024)

- Prompt engineering can be automated through APO
- Two approaches:
 - → Instruction Optimization
 - → Exemplar Optimization
- They are synergistic, but
- → EO is easier and more effective

System Prompts, Zhang et al. (2024)

What system prompts are

- System prompts are general instructions that are not task specific but are included in a prompt
- Some authors call them meta-instructions
- An example might be “Let’s think step by step”
- Recent studies show that the state-of-the-art prompt optimizer is ProTeGi
- Accordingly, Zhang et al. (2024) test a system prompt optimizer with it and find that the system prompt optimizer performs better, noting that system prompts are more generally applicable

Zhang et al. (2024), continued

- Zhang et al. (2024) cite recent research showing that combining multiple benchmarks in a single evaluation improves efficiency and alignment with human preferences, so they evaluate on a lot of task types
- Zhang et al. (2024) uses a genetic algorithm to iterate over system prompts (tens of thousands at each step), relying on removing mistakes from output at each iteration

Example of successful system prompt

Write an answer that makes the reader feel happy. Write like you are explaining. First establish the set of facts you know, then answer the question based only on those facts.

Ethical considerations

(We covered this last time, but a reminder doesn't hurt)

Admirably, Zhang et al. (2024) includes a section on ethical considerations. These include:

- energy consumption of the process
- possibility of stereotyping people, e.g., “act like a professor”
- crowdsourced annotations from WEIRD contexts
- need for research on model performance in new cultural and social contexts

Sagemaker Studio Lab

⟨ pause to look at the mistral tutorial ⟩

END

References

- Sahoo, Pranab, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. "A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications." <https://arxiv.org/abs/2402.07927>.
- Schulhoff, Sander, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, et al. 2024. "The Prompt Report: A Systematic Survey of Prompting Techniques." <https://arxiv.org/abs/2406.06608>.
- Wan, Xingchen, Ruoxi Sun, Hootan Nakhost, and Sercan O. Arik. 2024. "Teach Better or Show Smarter? On Instructions and Exemplars in Automatic Prompt Optimization." <https://arxiv.org/abs/2406.15708>.
- Zhang, Lechen, Tolga Ergen, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. 2024. "SPRIG: Improving Large Language Model Performance by System Prompt Optimization." <https://arxiv.org/abs/2410.14826>.

Colophon

This slideshow was produced using [quarto](#)

Fonts are *Roboto Light*, *Roboto Bold*, and *JetBrains Mono Nerd Font*